

# Nunchi nHYPE SY Audit Report

Dec 17, 2025





# Table of Contents

|                                                                                                                                                                                                                                                        |    |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----|
| Summary                                                                                                                                                                                                                                                | 2  |
| Overview                                                                                                                                                                                                                                               | 3  |
| Issues                                                                                                                                                                                                                                                 | 4  |
| [WP-M1] <code>StakingVaultManager.claimWithdraw()</code> and <code>batchClaimWithdraws()</code> may cause <code>SY_HYPE._deposit()</code> and <code>SY_HYPE.exchangeRate()</code> to revert, leading to potential DOS attacks on <code>SY_nHYPE</code> | 4  |
| [WP-L2] The contract name <code>SY_HYPE</code> is inconsistent with its file name <code>SY_nHYPE.sol</code> .                                                                                                                                          | 11 |
| [WP-L3] <code>previewDeposit()</code> should handle <code>minimumDepositAmount</code> and <code>stripUnsafePrecision</code> related logic to maintain consistency with the underlying SY implementation                                                | 13 |
| [WP-L4] <code>SY_HYPE</code> may not need or support receiving native HYPE token directly through <code>receive() external payable {}</code>                                                                                                           | 16 |
| [WP-G5] Code already implemented in <code>SYBaseUpgV2</code> , no need to implement again in inherited contracts                                                                                                                                       | 19 |
| [WP-N6] Consider adding a consistency check between <code>_nHype</code> and <code>stakingVaultManager.nHype()</code> .                                                                                                                                 | 23 |
| Appendix                                                                                                                                                                                                                                               | 24 |
| Disclaimer                                                                                                                                                                                                                                             | 25 |



## Summary

This report has been prepared for Nunchi smart contract, to discover issues and vulnerabilities in the source code of their Smart Contract as well as any contract dependencies that were not part of an officially recognized library. A comprehensive examination has been performed, utilizing Static Analysis and Manual Review techniques.

The auditing process pays special attention to the following considerations:

- Testing the smart contracts against both common and uncommon attack vectors.
- Assessing the codebase to ensure compliance with current best practices and industry standards.
- Ensuring contract logic meets the specifications and intentions of the client.
- Cross referencing contract structure and implementation against similar smart contracts produced by industry leaders.
- Thorough line-by-line manual review of the entire codebase by industry experts.



# Overview

## Project Summary

|              |                                                                                           |
|--------------|-------------------------------------------------------------------------------------------|
| Project Name | Nunchi                                                                                    |
| Codebase     | <a href="https://github.com/Nunchi-trade/nhype">https://github.com/Nunchi-trade/nhype</a> |
| Commit       | 2e4fcd6d7828225c137e47d2594b9e28e0896b4d                                                  |
| Language     | Solidity                                                                                  |

## Audit Summary

|                   |                                |
|-------------------|--------------------------------|
| Delivery Date     | Dec 17, 2025                   |
| Audit Methodology | Static Analysis, Manual Review |
| Total Issues      | 6                              |



[WP-M1] `StakingVaultManager.claimWithdraw()` and `batchClaimWithdraws()` may cause `SY_HYPE._deposit()` and `SY_HYPE.exchangeRate()` to revert, leading to potential DOS attacks on `SY_nHYPE`

Medium

## Issue Description

Users can deposit HYPE in advance (via `StakingVaultManager.deposit` ) and frequently call `StakingVaultManager.claimWithdraw()` and `batchClaimWithdraws()` , causing a DOS attack on `SY_nHYPE` .

This is because `SY_HYPE._deposit` and `SY_HYPE.exchangeRate` indirectly call `StakingVault.spotBalance` , which requires `block.number > lastSpotBalanceChangeBlockNumber` .

When `StakingVaultManager.claimWithdraw` and `StakingVaultManager.batchClaimWithdraws` are called, they update the `StakingVault.lastSpotBalanceChangeBlockNumber` field, causing `SY_HYPE._deposit` and `SY_HYPE.exchangeRate` to revert within the same block.

If `SY_HYPE.exchangeRate` reverts, it can cause `PendleYieldToken.transfer()`, `PendleMarketV3.swapExactPtForSy()`, `PendlePYLpOracle.getPtToSyRate()` and other functions to revert.

## Recommendation

Consider modifying the withdrawal logic in `StakingVaultManager` so that user operations (like `claimWithdraw()` and `batchClaimWithdraws()` ) no longer trigger async transfers. Instead, handle async fund transfers in batches.

Consider adding a new interface that completes async fund transfers for the entire batch at once after `block.timestamp > batch.finalizedAt + 7 days + claimWindowBuffer` .

## Reference Code

### `_deposit`

`SYnHYPE.sol`L42

```

42     function _deposit(address tokenIn, uint256 amountDeposited) internal override
      returns (uint256) {
43         // If depositing nHYPE directly, just return them (1:1)
44         if (tokenIn == yieldToken) {
45             return amountDeposited;
46         }
47
48         // Only accept HYPE (native) as other input
49         if (tokenIn != HYPE) {
50             revert Errors.SYInvalidTokenIn(tokenIn);
51         }
52
53         // Get nHYPE balance before deposit
54         uint256 balanceBefore = IERC20(yieldToken).balanceOf(address(this));
55
56         // Deposit HYPE to StakingVaultManager (will mint nHYPE to this contract)
57         stakingVaultManager.deposit{value: amountDeposited}();
58
59         // Return the amount of nHYPE received
60         uint256 nHypeReceived = IERC20(yieldToken).balanceOf(address(this)) -
      balanceBefore;
61         return nHypeReceived;
62     }

```

`StakingVaultManager.sol`L113

```

113     function deposit() external payable canDeposit whenNotPaused {
114         uint256 amountToDeposit = msg.value.stripUnsafePrecision();
115
116         // Mint nHYPE
117         // IMPORTANT: We need to make sure that we mint the nHYPE _before_
      transferring the HYPE to the staking vault,
118         // otherwise the exchange rate will be incorrect. We want the exchange
      rate to be calculated based on the total
119         // HYPE in the vault _before_ the deposit

```

```

120     uint256 amountToMint = HYPETonHYPE(amountToDeposit);
121     require(amountToMint > 0, ZeroAmount());
122     nHYPE.mint(msg.sender, amountToMint);
123
124     // Transfer HYPE to staking vault (HyperEVM -> HyperEVM)
125     if (amountToDeposit > 0) {
126         stakingVault.deposit{value: amountToDeposit}();
127     }
128
129     emit Deposit(msg.sender, amountToMint, amountToDeposit);
130 }

```

### StakingVaultManager.solL503

```

503     function HYPETonHYPE(uint256 hypeAmount) public view returns (uint256) {
504         return _HYPETonHYPE(hypeAmount, exchangeRate());
505     }

```

### exchangeRate

#### SY<sub>n</sub>HYPE.solL85

```

85     /// @notice Returns the exchange rate from StakingVaultManager
86     /// @return Exchange rate (HYPE per nHYPE) in 18 decimals
87     function exchangeRate() public view override returns (uint256) {
88         // Get the exchange rate from StakingVaultManager
89         // This represents how much HYPE 1 nHYPE is worth
90         return stakingVaultManager.exchangeRate();
91     }

```

### StakingVaultManager.exchangeRate

#### StakingVaultManager.solL529

```

529     function exchangeRate() public view returns (uint256) {
530         uint256 balance = totalBalance();
531         uint256 totalSupply = nHYPE.totalSupply();

```

```

532
533     // If we have no nHYPE in circulation, the exchange rate is 1
534     if (totalSupply == 0) {
535         return 1e18;
536     }
537
538     // If we have no HYPE in the vault, the exchange rate is 0
539     if (balance == 0) {
540         return 0;
541     }
542
543     return Math.mulDiv(balance, 1e18, totalSupply);
544 }
545
546 /// @inheritdoc IStakingVaultManager
547 function totalBalance() public view returns (uint256) {
548     // EVM + Spot + Staking account balances
549     uint256 accountBalances = stakingAccountBalance() + spotAccountBalance() +
    stakingVault.evmBalance();
550
551     // The total amount of HYPE that is reserved to be returned to users for
    withdraws, but is still in
552     // under the StakingVault accounts because they have not finished
    processing or been claimed
553     uint256 reservedHypeForWithdraws = totalHypeProcessed - totalHypeClaimed;
554
555     // This might happen right after a slash, before we're able to adjust the
    slashed exchange rate for the
556     // processed withdraws that are waiting for the 7-day withdraw period to
    pass. In practice, we would
557     // pause the contract in the case of a slash, but there could be a small
    window of time right after a
558     // slash where this could happen. So we throw an explicit error in this
    case.
559     require(accountBalances >= reservedHypeForWithdraws,
    AccountBalanceLessThanReservedHypeForWithdraws());
560
561     return accountBalances - reservedHypeForWithdraws;
562 }

```

```

574     function spotAccountBalance() public view returns (uint256) {
575         L1ReadLibrary.SpotBalance memory spotBalance =
stakingVault.spotBalance(stakingVault.HYPE_TOKEN_ID());
576         return spotBalance.total.to18Decimals();
577     }

```

## StakingVault.spotBalance

StakingVault.solL139

```

139     function spotBalance(uint64 tokenId) public view returns
(L1ReadLibrary.SpotBalance memory) {
140         // IMPORTANT: We enforce a one-block delay for reading the spot balance
after any spot
141         // balance changes. After an action occurs that changes the spot balance,
the L1Read
142         // precompile will no longer return an up-to-date balance.
143         require(block.number > lastSpotBalanceChangeBlockNumber,
CannotReadSpotBalanceUntilNextBlock());
144         return L1ReadLibrary.spotBalance(address(this), tokenId);
145     }

```

## Public Interface for Updating lastSpotBalanceChangeBlockNumber

StakingVaultManager.solL205

```

205     function claimWithdraw(uint256 withdrawId, address destination) public
whenNotPaused {
206         uint256 hypeAmount = _claimWithdraw(withdrawId);
207         if (hypeAmount.to8Decimals() > 0) {
208             stakingVault.spotSend(destination, stakingVault.HYPE_TOKEN_ID(),
hypeAmount.to8Decimals());
209         }
210     }
211
212     /// @inheritdoc IStakingVaultManager
213     function batchClaimWithdraws(uint256[] calldata withdrawIds, address
destination) public whenNotPaused {
214         require(withdrawIds.length > 0, ZeroAmount());

```

```

215     uint256 hypeAmounts = 0;
216     for (uint256 i = 0; i < withdrawIds.length; i++) {
217         hypeAmounts += _claimWithdraw(withdrawIds[i]);
218     }
219     if (hypeAmounts.to8Decimals() > 0) {
220         stakingVault.spotSend(destination, stakingVault.HYPE_TOKEN_ID(),
221             hypeAmounts.to8Decimals());
222     }

```

## StakingVault.solL89

```

89     function spotSend(address destination, uint64 token, uint64 weiAmount)
    external onlyManager whenNotPaused {
90         require(weiAmount > 0, ZeroAmount());
91
92         // Note: If the destination account doesn't exist on HyperCore, the
spotSend will silently fail
93         // and the HYPE will not actually be sent. We check the account exists
before making the call,
94         // so users don't lose their HYPE if their HyperCore account doesn't exist.
95         L1ReadLibrary.CoreUserExists memory coreUserExists =
        L1ReadLibrary.coreUserExists(destination);
96         require(coreUserExists.exists, CoreUserDoesNotExist(destination));
97
98         // Note: We don't expect to run into this case, but we're adding this check
for safety. The spotSend call will
99         // silently fail if the vault doesn't have enough HYPE, so we check the
balance before making the call.
100        uint256 _spotBalance = spotBalance(HYPE_TOKEN_ID).total;
101        require(_spotBalance >= weiAmount, InsufficientHYPEBalance());
102
103        CoreWriterLibrary.spotSend(destination, token, weiAmount);
104
105        lastSpotBalanceChangeBlockNumber = block.number;
106    }

```



## Status

✓ Fixed

[WP-L2] The contract name `SY_HYPE` is inconsistent with its file name `SY_nHYPE.sol` .

Low

## Issue Description

Consider renaming both the contract and the file to `PendleNHypeSY` to maintain consistency and follow the SY naming convention.

Additionally, instead of configuring the ERC20 symbol and name as parameters, consider hardcoding them to `SY Nunchi HYPE` and `SY-nHYPE` .

[https://github.com/Nunchi-trade/nhype/blob/1e0980bfef22d9a97e9e56c4d42977411efde7df/src/SY\\_nHYPE.sol](https://github.com/Nunchi-trade/nhype/blob/1e0980bfef22d9a97e9e56c4d42977411efde7df/src/SY_nHYPE.sol)

```

10  @@ 1,9 @@
11  /// @title SY_HYPE
12  /// @notice Standardized Yield wrapper for HYPE staking via StakingVaultManager
13  /// @dev Accepts HYPE deposits, returns nHYPE on redemption
14  contract SY_HYPE is SYBaseUpgV2 {
15  @@ 15,167 @@
168  }
```

```

104  /**
105   * @notice Create a pair of (PT, YT) from any SY and valid expiry. Anyone can
106   * create a yield contract
107   * @dev It's intentional to make expiry an uint32 to guard against fat fingers.
108   * uint32.max is year 2106
109   */
110  function createYieldContract(address SY, uint32 expiry, bool
111  doCacheIndexSameBlock)
112  external
113  returns (address PT, address YT)
114  {
```

```
@@ 112,116 @@  
117  
118     IStandardizedYield _SY = IStandardizedYield(SY);  
119  
120     (,, uint8 assetDecimals) = _SY.assetInfo();  
121  
122     string memory syCoreName = _SY.name().stripPrefix(SY_NAME_PREF);  
123     string memory syCoreSymbol = _SY.symbol().stripPrefix(SY_SYMBOL_PREF);  
124  
@@ 125,165 @@  
166     }
```

```
50     string private constant SY_SYMBOL_PREF = "SY-";  
51     string private constant SY_NAME_PREF = "SY ";
```

## Status

✓ Fixed

[WP-L3] `previewDeposit()` should handle `minimumDepositAmount` and `stripUnsafePrecision` related logic to maintain consistency with the underlying SY implementation

Low

## Issue Description

*SY<sub>n</sub>HYPE.sol* L97 – L112

```

97  function _previewDeposit(address tokenIn, uint256 amountTokenToDeposit) internal
    view override returns (uint256) {
98      // If depositing nHYPE directly, 1:1 conversion
99      if (tokenIn == yieldToken) {
100         return amountTokenToDeposit;
101     }
102
103     // For HYPE deposits, calculate nHYPE amount using exchange rate
104     if (tokenIn == HYPE) {
105         uint256 rate = stakingVaultManager.exchangeRate();
106         if (rate == 0) return 0;
107         // HYPE amount / exchange rate = nHYPE amount
108         return amountTokenToDeposit.divDown(rate);
109     }
110
111     revert Errors.SYInvalidTokenIn(tokenIn);
112 }

```

```

113 function deposit() external payable canDeposit whenNotPaused {
114     uint256 amountToDeposit = msg.value.stripUnsafePrecision();
115
116     // Mint nHYPE
117     // IMPORTANT: We need to make sure that we mint the nHYPE _before_
    transferring the HYPE to the staking vault,
118     // otherwise the exchange rate will be incorrect. We want the exchange rate to
    be calculated based on the total
119     // HYPE in the vault _before_ the deposit
120     uint256 amountToMint = HYPETonHYPE(amountToDeposit);
121     require(amountToMint > 0, ZeroAmount());

```

```

122     nHYPE.mint(msg.sender, amountToMint);
123
124     // Transfer HYPE to staking vault (HyperEVM -> HyperEVM)
125     if (amountToDeposit > 0) {
126         stakingVault.deposit{value: amountToDeposit}();
127     }
128
129     emit Deposit(msg.sender, amountToMint, amountToDeposit);
130 }

```

```

769 modifier canDeposit() {
770     _canDeposit();
771     _;
772 }
773
774 function _canDeposit() internal view {
775     require(msg.value >= minimumDepositAmount, BelowMinimumDepositAmount());
776 }

```

The rounding down functionality needs to be implemented to help the frontend calculate and prevent users from transferring unnecessary HYPE tokens.

Reference: PendleKinetiqKHYPESY.solL55-L65

```

55 function _previewDeposit(
56     address tokenIn,
57     uint256 amountTokenToDeposit
58 ) internal view virtual override returns (uint256 /*amountSharesOut*/) {
59     if (tokenIn == NATIVE) {
60         amountTokenToDeposit = _truncAmount(amountTokenToDeposit);
61         _validateStakeAmount(amountTokenToDeposit);
62         return
        IKinetiqStakingAccountant(STAKING_ACCOUNTANT).HYPEToKHYPE(amountTokenToDeposit);
63     }
64     return amountTokenToDeposit;
65 }

```

<https://github.com/pendle-finance/Pendle-SY-Public/blob/>



21ccfee6c24936fb73c1ae78d1a87c83b05f105c/contracts/core/StandardizedYield/  
implementations/Kinetiq/PendleKinetiqKHYPESY.sol#L99-L101

```
99  function _truncAmount(uint256 amount) internal pure virtual returns (uint256) {  
100      return amount - (amount % DEPOSIT_DENOM);  
101  }
```

## Status

✓ Fixed

## [WP-L4] `SY_HYPE` may not need or support receiving native HYPE token directly through `receive() external payable {}`

Low

### Issue Description

No code like `WNATIVE.withdraw()` that requires `receive() external payable {}` was found.

No code was found to handle native HYPE tokens received through `receive() external payable {}`.

It's possible that only `SY_HYPE.deposit()` needs to be payable.

[https://github.com/Nunchi-trade/nhype/blob/1e0980bfef22d9a97e9e56c4d42977411efde7df/src/SY\\_nHYPE.sol](https://github.com/Nunchi-trade/nhype/blob/1e0980bfef22d9a97e9e56c4d42977411efde7df/src/SY_nHYPE.sol)

```

10  @@ 1,9 @@
11  /// @title SY_HYPE
12  /// @notice Standardized Yield wrapper for HYPE staking via StakingVaultManager
13  /// @dev Accepts HYPE deposits, returns nHYPE on redemption
14  contract SY_HYPE is SYBaseUpgV2 {
15  @@ 15,36 @@
37
38  /// @notice Internal deposit function
39  /// @param tokenIn Either HYPE (native) or nHYPE
40  /// @param amountDeposited Amount to deposit
41  /// @return Amount of shares to mint
42  function _deposit(address tokenIn, uint256 amountDeposited) internal override
43  returns (uint256) {
44      // If depositing nHYPE directly, just return them (1:1)
45      if (tokenIn == yieldToken) {
46          return amountDeposited;
47      }
48
49      // Only accept HYPE (native) as other input
50      if (tokenIn != HYPE) {
51          revert Errors.SYInvalidTokenIn(tokenIn);
52      }
53  }

```

```

52
53     // Get nHYPE balance before deposit
54     uint256 balanceBefore = IERC20(yieldToken).balanceOf(address(this));
55
56     // Deposit HYPE to StakingVaultManager (will mint nHYPE to this contract)
57     stakingVaultManager.deposit{value: amountDeposited}();
58
59     // Return the amount of nHYPE received
60     uint256 nHypeReceived = IERC20(yieldToken).balanceOf(address(this)) -
balanceBefore;
61     return nHypeReceived;
62 }
63
64 /// @notice Internal redeem function - ONLY outputs nHYPE
65 /// @param receiver Address to receive the nHYPE
66 /// @param tokenOut Must be nHYPE
67 /// @param amountSharesToRedeem Amount of shares to burn
68 /// @return Amount of nHYPE transferred
69 function _redeem(address receiver, address tokenOut, uint256
amountSharesToRedeem)
70     internal
71     override
72     returns (uint256)
73 {
74     // Only nHYPE is supported as output (no direct HYPE redemption)
75     if (tokenOut != yieldToken) {
76         revert Errors.SYInvalidTokenOut(tokenOut);
77     }
78
79     // Transfer nHYPE to receiver (1:1 with shares)
80     _transferOut(yieldToken, receiver, amountSharesToRedeem);
81
82     return amountSharesToRedeem;
83 }
84
@@ 85,164 @@
165
166 /// @notice Receive function to accept HYPE
167 receive() external payable {}
168 }

```



## Status

✓ Fixed

## [WP-G5] Code already implemented in SYBaseUpgV2 , no need to implement again in inherited contracts

Gas

### Issue Description

The `receive()` function is already implemented in SYBaseUpgV2 :

```
166     /// @notice Receive function to accept HYPE
167     receive() external payable {}
```

`isValidTokenIn()` and `isValidTokenOut()` are already implemented in SYBaseUpgV2:

```
42     function _deposit(address tokenIn, uint256 amountDeposited) internal override
    returns (uint256) {
43         // If depositing nHYPE directly, just return them (1:1)
44         if (tokenIn == yieldToken) {
45             return amountDeposited;
46         }
47
48         // Only accept HYPE (native) as other input
49         if (tokenIn != HYPE) {
50             revert Errors.SYInvalidTokenIn(tokenIn);
51         }
52
53         // Get nHYPE balance before deposit
54         uint256 balanceBefore = IERC20(yieldToken).balanceOf(address(this));
55
56         // Deposit HYPE to StakingVaultManager (will mint nHYPE to this contract)
57         stakingVaultManager.deposit{value: amountDeposited}();
58
59         // Return the amount of nHYPE received
60         uint256 nHypeReceived = IERC20(yieldToken).balanceOf(address(this)) -
    balanceBefore;
61         return nHypeReceived;
62     }
63
64     /// @notice Internal redeem function - ONLY outputs nHYPE
```

```

65     /// @param receiver Address to receive the nHYPE
66     /// @param tokenOut Must be nHYPE
67     /// @param amountSharesToRedeem Amount of shares to burn
68     /// @return Amount of nHYPE transferred
69     function _redeem(address receiver, address tokenOut, uint256
amountSharesToRedeem)
70         internal
71         override
72         returns (uint256)
73     {
74         // Only nHYPE is supported as output (no direct HYPE redemption)
75         if (tokenOut != yieldToken) {
76             revert Errors.SYInvalidTokenOut(tokenOut);
77         }
78
79         // Transfer nHYPE to receiver (1:1 with shares)
80         _transferOut(yieldToken, receiver, amountSharesToRedeem);
81
82         return amountSharesToRedeem;
83     }

```

```

97     function _previewDeposit(address tokenIn, uint256 amountTokenToDeposit)
internal view override returns (uint256) {
98         // If depositing nHYPE directly, 1:1 conversion
99         if (tokenIn == yieldToken) {
100             return amountTokenToDeposit;
101         }
102
103         // For HYPE deposits, calculate nHYPE amount using exchange rate
104         if (tokenIn == HYPE) {
105             uint256 rate = stakingVaultManager.exchangeRate();
106             if (rate == 0) return 0;
107             // HYPE amount / exchange rate = nHYPE amount
108             return amountTokenToDeposit.divDown(rate);
109         }
110
111         revert Errors.SYInvalidTokenIn(tokenIn);
112     }
113
114     /// @notice Preview redeem amount
115     /// @param tokenOut Token to receive (must be nHYPE)

```

```

116     /// @param amountSharesToRedeem Amount of shares to redeem
117     /// @return Amount of tokens that would be received
118     function _previewRedeem(address tokenOut, uint256 amountSharesToRedeem)
119         internal pure override returns (uint256) {
120         // Only nHYPE supported as output
121         if (tokenOut != yieldToken) {
122             revert Errors.SYInvalidTokenOut(tokenOut);
123         }
124         // 1:1 redemption for nHYPE
125         return amountSharesToRedeem;
126     }

```

SYBaseUpgV2:

<https://github.com/pendle-finance/Pendle-SY-Public/blob/21ccfee6c24936fb73c1ae78d1a87c83b05f105c/contracts/core/StandardizedYield/v2/SYBaseUpgV2.sol#L31-L32>

```

31     /// solhint-disable no-empty-blocks
32     receive() external payable {}

```

<https://github.com/pendle-finance/Pendle-SY-Public/blob/21ccfee6c24936fb73c1ae78d1a87c83b05f105c/contracts/core/StandardizedYield/v2/SYBaseUpgV2.sol#L41-L81>

```

41     function deposit(
42         address receiver,
43         address tokenIn,
44         uint256 amountTokenToDeposit,
45         uint256 minSharesOut
46     ) external payable nonReentrant returns (uint256 amountSharesOut) {
47         if (!isValidTokenIn(tokenIn)) revert Errors.SYInvalidTokenIn(tokenIn);
48         @@ 48,56 @@
49     }
50
51     /**
52     * @dev See {ISStandardizedYield-redeem}

```

```

61     */
62     function redeem(
63         address receiver,
64         uint256 amountSharesToRedeem,
65         address tokenOut,
66         uint256 minTokenOut,
67         bool burnFromInternalBalance
68     ) external nonReentrant returns (uint256 amountTokenOut) {
69         if (!isValidTokenOut(tokenOut)) revert Errors.SYInvalidTokenOut(tokenOut);
70         @@ 70,80 @@
81     }

```

<https://github.com/pendle-finance/Pendle-SY-Public/blob/21ccfee6c24936fb73c1ae78d1a87c83b05f105c/contracts/core/StandardizedYield/v2/SYBaseUpgV2.sol#L149-L163>

```

149 function previewDeposit(
150     address tokenIn,
151     uint256 amountTokenToDeposit
152 ) external view virtual returns (uint256 amountSharesOut) {
153     if (!isValidTokenIn(tokenIn)) revert Errors.SYInvalidTokenIn(tokenIn);
154     return _previewDeposit(tokenIn, amountTokenToDeposit);
155 }
156
157 function previewRedeem(
158     address tokenOut,
159     uint256 amountSharesToRedeem
160 ) external view virtual returns (uint256 amountTokenOut) {
161     if (!isValidTokenOut(tokenOut)) revert Errors.SYInvalidTokenOut(tokenOut);
162     return _previewRedeem(tokenOut, amountSharesToRedeem);
163 }

```

## Status

✓ Fixed

[WP-N6] Consider adding a consistency check between `_nHype` and `stakingVaultManager.nHype()` .

## Issue Description

Or, consider using the queried value from `stakingVaultManager.nHype()` directly instead of setting the value with a parameter to avoid configuration errors.

```
26     constructor(address _stakingVaultManager, address _nHype) SYBaseUpgV2(_nHype)
    {
27         stakingVaultManager = IStakingVaultManager(_stakingVaultManager);
28     }
```

```
17     /// forge-lint: disable-next-line(mixed-case-variable)
18     nHYPE public nHYPE;
```

## Status

✓ Fixed

# Appendix

## Timeliness of content

The content contained in the report is current as of the date appearing on the report and is subject to change without notice, unless indicated otherwise by WatchPug; however, WatchPug does not guarantee or warrant the accuracy, timeliness, or completeness of any report you access using the internet or other means, and assumes no obligation to update any information following publication.



## Disclaimer

This report is based on the scope of materials and documentation provided for a limited review at the time provided. Results may not be complete nor inclusive of all vulnerabilities. The review and this report are provided on an as-is, where-is, and as-available basis. You agree that your access and/or use, including but not limited to any associated services, products, protocols, platforms, content, and materials, will be at your sole risk. Smart Contract technology remains under development and is subject to unknown risks and flaws. The review does not extend to the compiler layer, or any other areas beyond the programming language, or other programming aspects that could present security risks. A report does not indicate the endorsement of any particular project or team, nor guarantee its security. No third party should rely on the reports in any way, including for the purpose of making any decisions to buy or sell a product, service or any other asset. To the fullest extent permitted by law, we disclaim all warranties, expressed or implied, in connection with this report, its content, and the related services and products and your use thereof, including, without limitation, the implied warranties of merchantability, fitness for a particular purpose, and non-infringement. We do not warrant, endorse, guarantee, or assume responsibility for any product or service advertised or offered by a third party through the product, any open source or third-party software, code, libraries, materials, or information linked to, called by, referenced by or accessible through the report, its content, and the related services and products, any hyperlinked websites, any websites or mobile applications appearing on any advertising, and we will not be a party to or in any way be responsible for monitoring any transaction between you and any third-party providers of products or services. As with the purchase or use of a product or service through any medium or in any environment, you should use your best judgment and exercise caution where appropriate. FOR AVOIDANCE OF DOUBT, THE REPORT, ITS CONTENT, ACCESS, AND/OR USAGE THEREOF, INCLUDING ANY ASSOCIATED SERVICES OR MATERIALS, SHALL NOT BE CONSIDERED OR RELIED UPON AS ANY FORM OF FINANCIAL, INVESTMENT, TAX, LEGAL, REGULATORY, OR OTHER ADVICE.